

TRABAJO PRÁCTICO

Programación I.



UNIVERSIDAD GENERAL SARMIENTO

Comisión 01
Grupo nro. 11

Estudiantes:

Correa María Luz | correa.luz.maria2015@gmail.com | DNI: 46946205

Valderrey Eduardo | eduar.vr.e@gmail.com | DNI: 95981566

Villaverde Giuliana | giulianvillaverde@gmail.com | DNI: 46830077

SUPER ELIZABETH SIS

El presente trabajo práctico consiste en el diseño, arquitectura e implementación de un videojuego en Java, usando la librería *Entorno*. La idea del juego está inspirada en Super Mario Bros, donde el objetivo principal es controlar a la princesa a lo largo de un mapa que avanza hacia la derecha, saltando entre islas flotantes, esquivando, o eliminando enemigos voladores y cuidando de no caer al vacío, ni quedarse sin vidas antes de tocar el castillo final para poder rescatar a Mario.

Descripción de las clases implementadas:

A continuación, explicaremos de forma breve cada clase implementada, sus variables y cuáles son sus funciones principales.

1. Clase Juego

Es el “corazón” del juego. Se encarga de coordinar el menú de inicio, el movimiento de las cosas, el conteo de vidas, las condiciones de victoria y derrota, el sistema de enemigos, las recompensas, la batalla contra el jefe final y de dibujar todo en pantalla en cada instante.

Variables:

- entorno: es el objeto que permite renderizar imágenes, figuras y leer el teclado o mouse.
- Princesa, castillo, fondo: las instancias únicas de los elementos fijos del mapa.
- Islas: colección o arreglo de estructuras de tipo Isla que componen el mapa.
- Enemigos y enemigos2: dos arreglos separados para manejar las tandas de rivales voladores.
- Proyectil, explosion: controlan el único disparo en pantalla y su correspondiente animación al impactar.
- Corazones, pociones: arreglos para la barra de vida y los ítems de regalo en el mapa.
- Vidas, juegoTerminado, victoria, menuActivo: booleanos y enteros que guardan el estado actual de la partida.

-anchoTotalMapa: define la longitud total del escenario y limita la generación de plataformas y la ubicación del castillo.

-contadorSpawn, intervaloMinEnemigos, minEnemigosPantalla: variables que controlan la generación automática de enemigos y la cantidad mínima presente en pantalla.

-tiempoReaparecer, reapareciendo, primeraVez: administran la reaparición de la princesa después de perder una vida y su ubicación inicial al comenzar la partida.

-tiempoExplosion: controla el tiempo de reutilización de la habilidad de explosión.

-enemigosEliminados, enemigosParaItem: registran la cantidad de enemigos derrotados y determinan cuándo debe aparecer una poción de vida como recompensa.

-fondoMenu, botonJugar, botonReiniciar, titulo, gameOverImage, victoriaImage: imágenes utilizadas para el menú principal y las pantallas de victoria o derrota.

-botonJugarX, botonJugarY, botonReiniciarX, botonReiniciarY: coordenadas de los botones interactivos del juego.

-botonAncho, botonAlto: dimensiones de las áreas de clic utilizadas para detectar la interacción del jugador.

-Jefe, proyectilesJefe, enPeleaJefe: variables encargadas de administrar el combate final contra el jefe, sus proyectiles y el estado de la batalla.

Métodos principales:

-Tick(): bucle principal que corre continuamente. Lee las teclas y el mouse, mueve a los personajes, aplica la gravedad, administra el sistema de enemigos, controla la pelea contra el jefe y revisa todas las colisiones del juego.

-moverNivel(): mueve las islas, enemigos, pociones, castillo, jefe final y el fondo hacia atrás cuando la princesa avanza hacia la derecha, creando el efecto de desplazamiento horizontal del escenario.

-reiniciarJuego() / iniciarPartida(): resetean todas las variables, eliminan enemigos y proyectiles, regeneran el mapa y preparan una nueva partida desde cero.

-dibujarMenu(): muestra la pantalla principal con el fondo, el título y el botón para comenzar el juego.

-dibujarFinJuego(): renderiza la pantalla de victoria o derrota y permite reiniciar la partida mediante un botón.

- dibujarTodo(): centraliza el dibujado de todos los elementos activos del juego, incluyendo fondo, plataformas, enemigos, proyectiles, objetos de curación, jefe final e interfaz de usuario.
- actualizarCorazones(): sincroniza la cantidad de corazones visibles con las vidas actuales de la princesa.
- iniciarReaparicion(): activa el proceso de reaparición luego de recibir daño o caer al vacío.
- generarPocion(): crea una poción de vida cuando el jugador elimina la cantidad necesaria de enemigos.
- iniciarPeleaJefe(): transforma el escenario para la batalla final, elimina a los enemigos normales, cambia el fondo, modifica las plataformas y activa al jefe final.
- hayColisionAlSpawn(): verifica que los enemigos nuevos no aparezcan demasiado cerca de otros enemigos o de las plataformas.
- colisionPrincesa(): conjunto de métodos que detectan y resuelven las colisiones de la princesa con plataformas, enemigos, enemigos especiales y proyectiles del jefe.
- colisionProyectil(): controla los impactos de los proyectiles de la princesa sobre enemigos comunes, enemigos especiales y el jefe final.
- colisionExplosion(): detecta los daños provocados por la explosión especial sobre los enemigos cercanos.

2. Clase princesa

Personaje controlado por el jugador.

Variables:

- X, y: posición central de la princesa en pantalla.
- VelocidadY: la velocidad vertical que cambia dinámicamente debido a los saltos o la gravedad.
- Salto, caída: booleanos de control físico para saber si el personaje está subiendo o cayendo.
- Ciclos: contador interno entero que mide cuanto tiempo dura el impulso hacia arriba del salto.
- ImagenIzquierda, imagenDerecha: imágenes animadas (GIF) correspondientes al sentido donde camina.

- MirandoIzquierda: booleano para saber que imagen dibujar en la pantalla.
- Arriba, abajo, izquierda, derecha: coordenadas de sus cuatro extremos para los choques.

Métodos principales:

- moverse(velocidad): Cambia la posición horizontal según las teclas “A” o “D” (o flechas) sin dejar que se salga de la pantalla.
- movVertical() / iniciarSalto(): Manejan la fuerza para subir cuando salta y la caída constante simulando la gravedad.
- actualColis(): Actualiza los bordes del personaje (arriba, abajo, izquierda, derecha) para detectar choques.

3. Clase isla

Son las plataformas flotantes donde la princesa se puede parar para no caer al vacío.

Variables:

- X, y: posición en el mapa.
- Escala: factor numérico para ajustar el tamaño visual según el tipo.
- Tipo: un entero que define las propiedades de la plataforma: 1 para isla grande, 2 para la mediana, 3 para la chica.
- Arriba, abajo, izquierda. Derecha: límites correspondientes de la plataforma que actúan como superficie sólida.

Métodos principales:

- dibujar () para mostrar plataforma en pantalla usando la escala adecuada.
- actualColis() para reubicar sus bordes cuando el mapa se mueve.

4. Clase enemigo

Primer tipo de rival volador. Se genera en los bordes y avanza en línea recta de un lado a otro cruzando la pantalla para dañar a la princesa.

Variables:

- X, y: ubicación espacial en el entorno.

- Velocidad: factor que determina la rapidez y el sentido del recorrido (positivo si va a la derecha, negativo a la izquierda)
- Activo: booleano que indica si el enemigo está vivo y debe procesarse.
- Imagen: carga un GIF diferente según la dirección de su velocidad.
- LimiteIzquierdo, limiteDerecho: márgenes invisibles fuera de la pantalla que sirven para detectar cuando se fue del mapa.
- Arriba, abajo, derecha, izquierda: bordes exteriores para detectar choques.

Métodos principales:

- mover (): para avanzar de un lado a otro.
- actualizarColisiones (): para sus bordes
- estaFueraDePantalla. () para avisar si el enemigo ya se fue de la pantalla para poder borrarlo.

5. Clase enemigo2

Es el segundo tipo de rival. Quita más salud si toca la princesa, quitándole dos vidas y tiene una mecánica de vuelo más compleja en forma de ondas.

Variables:

- yBase: guarda la altura original en la que apareció el enemigo, sirviendo como eje central para su balanceo.
- amplitudOndulacion: el rango en pixeles que determina que tan arriba y abajo se mueve (fijado en 18)
- frecuenciaOndulacion : la velocidad con la que realiza el ciclo de subida y bajada (fijada en 0.06).
- tickContador : entero que suma 1 en cada ciclo para hacer avanzar a la fórmula matemática del movimiento.
- Comparte con enemigo: variables de posición, velocidad, escala, estado activo, imágenes específicas, límites de pantalla y variables de bordes.

Métodos principales:

- mover(): modifica las variables x de forma recta, pero altera las variables y aplicando la función trigonométrica Math.sin multiplicada por la amplitud sobre el tickContador. Esto logra que el enemigo vuele haciendo ondas.
- dibujar (entorno e) y estaFueraDePantalla (): cumplen la misma función de dibujado y descarte que en el enemigo común.

6. Clase proyectilPrincesa

Disparo que tira la princesa con el botón izquierdo del mouse para matar enemigos.

Variables:

- x, y: posición del proyectil en el aire.
- velocidadX, velocidadY: los componentes del vector de movimiento. Se calculan multiplicando los valores de dirección por 10. para darle velocidad al disparo.
- Activo: booleano de estado. Si choca con alto o sale del mapa, pasa a falso.
- Arriba, abajo, izquierda, derecha: caja de colisión para destruir a los enemigos.

Métodos principales:

- mover (): actualiza la posición sumando las velocidades calculadas en ambos ejes simultáneamente, logrando un desplazamiento en línea recta diagonal.
- Dibujar (entorno e): muestro el proyectil en pantalla mientras siga activo.
- EstaFueraDePantalla(): revisa si el disparo supero los limites físicos de la pantalla de juego para poder desactivarlo y limpiar el espacio.

7. Clase explosionPrincesa

Es un efecto visual que se genera en el lugar exacto donde un proyectil impacta u destruye a cualquiera de los enemigos.

Variables:

- X, y: ubicación donde ocurre el impacto.
- Escala: empieza en 0 y va aumentando para dar la sensación de que la explosión se agrande.
- Tiempo: contador numérico rico que mide la duración del efecto.

- Fin: booleano que se vuelve verdadero cuando la animación se completa y el objeto debe borrarse.

Métodos principales:

- Dibujar (entorno e): dibuja la imagen centrada usando la escala actual.
- Mover(double velocidadScroll): incrementa el contador de tiempo y aumenta el valor de la escala para agrandar la imagen. Resta la velocidad de scroll a su x para que la explosión se mueva hacia atrás junto con el mapa si la princesa sigue avanzando. Si el tiempo llega a su límite, activa la variable *fin*.
- ActualizarColisiones: redimensiona los bordes en cada frame ya que el alto y el ancho cambian constantemente debido al aumento de la escala.

8. Clase castillo

Es la estructura que se encuentra en el extremo derecho del nivel. Representa la meta final del juego.

Variables:

- x, y: Ubicación fija al final del mapa.
- imagen: Gráfico del castillo (castillo.png).
- activo: Estado booleano del elemento.
- arriba, abajo, izquierda, derecha: Límites de contacto que la clase Juego analiza para activar la victoria.

Métodos principales:

- dibujar(Entorno e): Dibuja el castillo en pantalla cuando el jugador avanza lo suficiente en el mapa como para verlo.
- actualizarColisiones(): Mueve el castillo al mismo ritmo que se desplaza todo el nivel por el scroll.

9. Clase corazon

Es un elemento visual que sirve para mostrar cuantas vidas le quedan a la princesa.

Variables:

- x, y: Coordenadas fijas en la esquina superior de la interfaz.

- imagen: Archivo de imagen animado (corazon.gif).
- activo: Booleano que indica si la vida está disponible. Si la princesa sufre daño, este estado se apaga.

Métodos principales:

- dibujar(Entorno e): Si el corazón está activo, muestra el GIF animado. Si ocurre un fallo y la imagen no carga, el método tiene un respaldo que dibuja un cuadrado de color rojo para evitar errores visuales.
- desactivar(): Cambia el estado de activo a falso para que deje de dibujarse cuando el jugador pierde una vida.

10. Clase pocionVida

Es un ítem de recompensa que aparece flotando en el escenario de manera automática cuando el jugador elimina una cantidad específica de enemigos. Al tocarla, le devuelve una vida a la princesa.

Variables:

- x, y: Ubicación en el mapa donde se genera el ítem.
- imagen: Archivo gráfico de la poción (pocion.png).
- activo: Booleano que indica si la poción está en el mapa esperando ser recogida.
- arriba, abajo, izquierda, derecha: Bordes de choque para detectar si la princesa pasa por encima de ella.

Métodos principales:

- dibujar(Entorno e): Muestra la poción flotando en el escenario mientras no haya sido agarrada.
- actualizarColisiones(): Desplaza sus bordes de colisión en el eje X para acompañar el scroll general del mapa.
- desactivar(): Apaga el estado activo para que la poción desaparezca del juego una vez que cumple su función de curar.

11. Clase fondo

Se encarga de renderizar la imagen del paisaje trasero del juego y de gestionar un efecto infinito para que el fondo nunca se corte.

Variables:

- x, y: Posición central del fondo.
- imagenFondo: Archivo de imagen del paisaje (fondonuevo.png).
- anchoImagen: Guarda la medida exacta de ancho que tiene la imagen escalada para saber cuándo termina el dibujo.

Métodos principales:

- dibujar(): Renderiza el fondo principal. Tiene una lógica que controla los bordes de la pantalla: si el fondo se desplaza por el scroll y deja ver un hueco vacío a los costados, dibuja automáticamente una copia exacta de la imagen pegada justo al lado (a la izquierda o a la derecha). Esto evita que el jugador vea un fondo negro en pantallas anchas o recorridos largos.

12. Clase JefeFinal

Es el enemigo principal que aparece al final del nivel. Representa el último desafío antes de alcanzar la victoria. Posee varias vidas, puede atacar periódicamente a la princesa y muestra una barra de corazones al igual que la princesa.

Variables:

- x, y: posición central del jefe en el escenario.
- ancho, alto: dimensiones utilizadas para el cálculo de colisiones.
- escala: factor que ajusta el tamaño visual de la imagen.
- vidas: cantidad actual de vidas del jefe.
- vidasMaximas: cantidad máxima de vidas con la que inicia el combate.
- imagen: imagen del jefe final (JefeFinal.png).
- activo: booleano que indica si el jefe sigue vivo o ya fue derrotado.
- arriba, abajo, izquierda, derecha: límites de colisión utilizados para detectar impactos de proyectiles y contacto con otros objetos.
- contadorAtaque: contador interno que aumenta en cada ciclo del juego para controlar cuándo puede volver a atacar.

-intervaloAtaque: cantidad de ciclos que deben transcurrir entre un ataque y otro.

-corazonesJefe: arreglo de objetos Corazon que representa visualmente las vidas restantes del jefe en la interfaz.

Métodos principales:

-dibujar(Entorno e): muestra la imagen del jefe final en pantalla mientras permanezca activo.

-dibujarCorazones(Entorno e): actualiza y dibuja los corazones correspondientes a las vidas actuales del jefe.

-actualizarColisiones(): recalcula los límites superior, inferior, izquierdo y derecho para mantener actualizada la detección de colisiones.

-recibirDanio(): reduce una vida cada vez que el jefe recibe un impacto. Si las vidas llegan a cero, cambia el estado activo a falso indicando que fue derrotado.

-puedeAtacar(): controla el tiempo entre ataques. Devuelve verdadero cuando se alcanza el intervalo configurado y reinicia el contador.

-moverHacia(double targetX): desplaza lentamente al jefe en dirección a una posición objetivo sobre el eje horizontal.

-isActivo(): devuelve el estado actual del jefe para verificar si continúa participando en la partida o ya fue eliminado.

13. Clase ProyectoilJefe

Es el proyectil que dispara el jefe final durante el combate. Se dirige automáticamente hacia la posición donde se encuentra la princesa en el momento del disparo y continúa avanzando en línea recta hasta impactar o salir de la pantalla.

Variables:

-x, y: posición actual del proyectil en el escenario.

-rotacion: ángulo de orientación utilizado para que la imagen apunte en la dirección de movimiento.

-velocidadX, velocidadY: componentes horizontal y vertical de la velocidad calculadas a partir de la posición del objetivo.

-ancho, alto: dimensiones utilizadas para la detección de colisiones.

- activo: booleano que indica si el proyectil sigue existiendo en el juego.
- anchoPantalla, altoPantalla: dimensiones del entorno utilizadas para verificar si el proyectil abandonó los límites visibles.
- arriba, abajo, izquierda, derecha: bordes exteriores que conforman la caja de colisión del proyectil.
- imagen: archivo gráfico del disparo enemigo (proyectilJefe.png).
- escala: factor que determina el tamaño visual con el que se dibuja el proyectil.

Métodos principales:

- mover(): actualiza la posición del proyectil sumando los valores de velocidad horizontal y vertical, permitiendo que avance en línea recta hacia la dirección calculada al ser creado.
- dibujar(Entorno e): muestra el proyectil en pantalla aplicando la rotación correspondiente para que la imagen apunte hacia donde se desplaza.
- actualizarColisiones(): recalcula los límites superior, inferior, izquierdo y derecho para mantener actualizada la detección de impactos.
- estaFueraDePantalla(): verifica si el proyectil salió de los límites del entorno de juego. Si esto ocurre, puede ser eliminado para liberar espacio y evitar procesamiento innecesario.

Funcionamiento general:

- Cuando el jefe realiza un ataque, el constructor calcula la distancia entre la posición inicial del proyectil y la ubicación actual de la princesa. A partir de esos datos genera un vector de dirección normalizado y establece las velocidades horizontal y vertical.
- Gracias a este cálculo, el disparo viaja directamente hacia el objetivo manteniendo una velocidad constante durante todo su recorrido.

Problemas encontrados y decisiones tomadas.

- Cómo hacer el movimiento de la pantalla (Scroll):

Problema: El enunciado pide que la princesa empiece en el medio y que el mapa avance hacia atrás si ella va a la derecha, pero sin que ella pase los límites de la pantalla. Si movíamos solo a la princesa, nos quedábamos sin pantalla muy rápidamente.

Solución: En la clase Juego, programamos que, si la princesa pasa de los dos tercios de la pantalla, en vez de mover su x, le restamos velocidad a las x de todas las islas, del castillo, de las pociones y de los enemigos. Así la princesa parece quedarse en su lugar mientras todo el nivel se mueve hacia atrás.

- El tamaño de las islas al escalarlas con una sola imagen:

Problema: Al principio intentamos usar una única imagen de isla para todo el juego. Para hacer los tres tamaños que pide el enunciado (chica, mediana y grande), cambiábamos el valor de la variable escala. El problema fue que las islas más grandes quedaban muy gigantes, borrosas y deformadas, perdiendo calidad visual y arruinando las cajas de colisión.

Solución: Decidimos crear tres archivos de imagen separados (islaGrande.png, islaMediana.png y islaChica.png) editados de antemano con sus proporciones correctas. En el constructor de la clase Isla, agregamos la variable tipo y pusimos una estructura *if-else* que carga la imagen y escala correspondiente según el tamaño que necesitemos para esa plataforma.

Implementación.

Este fragmento determina, para cada isla, si existe colisión con la princesa.

El código recorre todas las islas disponibles, calcula si la princesa y una de ellas comparten el mismo rango en el eje X. Esto determina si ella está posicionada, lateralmente, sobre o debajo de la isla.

Si la princesa se encuentra sobre una isla se detiene su caída y se cancela el estado de salto. Si la princesa salta y golpea una isla, se corrige su posición debajo de la misma y se anula su impulso vertical.

Si tras revisar todas las islas no se detecta colisión se activa el estado de caída libre para la princesa.

```

279
280 // Colisión de la princesa con islas
281 boolean enIsla = false;
282 for (Isla[] fila : this.islas) {
283     for (Isla isla : fila) {
284         if (isla != null) {
285             boolean colisionHorizontal = this.princesa.derecha > isla.izquierda
286                 && this.princesa.izquierda < isla.derecha;
287
288             double distanciaVertical = Math.abs(this.princesa.abajo - isla.arriba);
289
290             if (distanciaVertical < 15 && colisionHorizontal && this.princesa.velocidadY >= 0) {
291                 this.princesa.y = isla.arriba - this.princesa.alto / 2;
292                 this.princesa.velocidadY = 0;
293                 this.princesa.salto = false;
294                 this.princesa.caida = false;
295                 this.princesa.actualColis();
296                 enIsla = true;
297                 break;
298             }
299
300             distanciaVertical = Math.abs(this.princesa.arriba - isla.abajo);
301             if (distanciaVertical < 15 && colisionHorizontal && this.princesa.velocidadY < 0) {
302                 this.princesa.y = isla.abajo + this.princesa.alto / 2;
303                 this.princesa.velocidadY = 0;
304                 this.princesa.salto = false;
305                 this.princesa.ciclos = 0;
306                 this.princesa.actualColis();
307             }
308         }
309     }
310     if (enIsla)
311         break;
312 }
313
314 if (!enIsla) {
315     this.princesa.caida = true;
316 }
317

```

Conclusión.

Hacer este trabajo práctico nos sirvió mucho para entender cómo funciona la programación orientada a objetos en un caso real como un videojuego. Lo que más nos costó, pero a la vez nos dejó más aprendizaje, fue trabajar con las limitaciones de los arreglos fijos para elementos dinámicos como los enemigos. Tuvimos que ser muy ordenados controlando qué espacios del arreglo estaban vacíos con null y cuáles tenían objetos activos para evitar errores en pleno juego. También estuvo bueno tener que resolver matemática a través del salto, la gravedad y el movimiento del escenario en conjunto, logrando un juego divertido, funcional y que cumple con todos los requisitos pedidos, además de un menú para tener un buen diseño.